

计算机视觉 课程实验报告

学号：202300130183	姓名：宋浩宇	
-----------------	--------	--

实验题目：实验 1：图像基本操作

实验过程中遇到和解决的问题：

（记录实验过程中遇到的问题，以及解决过程和实验结果。可以适当配以关键代码辅助说明，但不要大段贴代码。）

问题 1: Opencv 环境配置时选择使用 cmake 从源码构建，但默认 configure 下是由 msvc 编译的，我原本的编译环境是通过 g++ 编译，二者使用的多线程模型不同（一个是 win32 一个是 posix），怎么解决？

解决 1: 更改 vscode 的 task.json 配置文件，通过调用 msvc 的 cl.exe 编译器来对源文件进行编译。

问题 2: 接上一个问题，msvc 需要特殊的环境变量配置，以及，我们想要成功 include 和链接 lib 需要配置路径，怎么办？

解决 2: 在编译器参数配置部分将 opencv_world480.lib 和 include 文件夹添加到搜索路径中来让编译器可以将 opencv 纳入编译过程。具体而言，我们有如下的 task.json:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "MSVC Build OpenCV",
      "type": "shell",
      "command": "cl.exe",
      "args": [
        "/nologo",
        "/EHsc",
        "/std:c++17",
        "/Zi",
        "${file}",
        "/Fe:${fileDirname}\\${fileBasenameNoExtension}.exe",
        "/I",
        "F:\\library",
        "/I",
        "D:\\OpenCV\\opencv\\build\\include",
        "/link",
        "/LIBPATH:\"D:\\OpenCV\\opencv\\build\\x64\\vc16\\lib\\\"",
        "opencv_world480.lib"
      ]
    }
  ]
}
```

```

        "options": {
            "cwd": "${fileDirname}",
            "shell": {
                "executable": "C:\\Windows\\System32\\cmd.exe",
                "args": [
                    "/d",
                    "/c",
                    "call \"F:\\Visual
Studio\\BuildTools\\VC\\Auxiliary\\Build\\vcvars64.bat\" &&"
                ]
            }
        },
        "problemMatcher": [
            "$msCompile"
        ],
        "group": {
            "kind": "build",
            "isDefault": true
        }
    }
}

```

问题 3: 我们构建的程序依赖于 opencv_world480.dll, 有什么解决方法?

解决 3: 最简单的方法是把 opencv_world480.dll 复制到当前文件夹下, 但这还是太麻烦, 直接通过修改环境变量来将 opencv 构建产物中的 opencv_world480.dll 所在路径添加到 PATH 中方便程序运行时的链接。

问题 4: 在 alpha 混合部分, 读入的背景图片无法提前确认是三通道还是四通道数据, 且 size 和前景图不一致, 这导致了我拿来测试的图片无法正确 split 和 merge.

解决 4: 统一格式化导入的图片, 将 Mat 数据均设为四通道, 对于三通道的图片, 为他添加一个全 255 的 alpha 通道 (第四通道), 且在 alphaBlend 之前先将背景图 resize.

```

// 检查是否为 4 通道
if (src.channels() != 4) {
    cout << path << "不是 4 通道图像" << endl;
    vector<Mat> channels;
    split(src, channels);
    Mat alpha = Mat::zeros(src.size(), CV_8UC1);
    alpha.setTo(Scalar(255));
    channels.push_back(alpha);
    merge(channels, src);
}

```

在混合函数中

```

// 调整背景图大小, 使其和前景图一致

```

```
resize(back, back, front.size());
```

问题 5: Alpha 混合阶段 multiply 函数报错, 且准确来说, 是 Windows 给了错误码 0xF000000C, 这是一个典型的内存访问错误, 如何定位和解决。

解决 5: 通过调试器打断点找到出错的函数是 mul, 根据 Windows 返回的错误码猜测应该是内存访问越界, 这大概率会出现在矩阵乘法维度不匹配里。随后定位到是我们实际混合是用的背景图是四通道的, 而我们前景图经过提取只留下了三通道, 只需要对背景图进行相同操作, 将 bgr 三通道提取出来即可。

```
{  
    vector<Mat> channels;  
    split(back, channels);  
    Mat b = channels[0];  
    Mat g = channels[1];  
    Mat r = channels[2];  
    back_bgrChannels = { b, g, r };  
}  
Mat backBGRChannels;  
merge(back_bgrChannels, backBGRChannels);
```

实验结果分析:

实验 1.1: 图像加载、显示



图片被正确的 imread 进来, 并且 imshow 和 waitkey 也正常, 这一部分实验没有问题。

实验 1.2: 图像合成

我们试图合并这两个图像:

前景图：



背景图：



通过我们写的 alpha 混合，得到结果：



从结果上看，alpha 混合的效果符合预期，且程序正常退出（返回值 0）。

总结：本次实验比较简单，困难的点都在环境配置和熟悉 opencv 上。最终结果则是将实验要求均成功完成。